

COMP90038: Assignment 2 Report

Aaditya Sharma
Student ID: 1814346

May 24, 2026

1 Experiment Environment

In order to ensure the reproducibility of these experiments, the hardware and software specifications of the environment are mentioned below:

- **Operating System:** Windows 11 Home, 64-bit
- **CPU:** 13th Gen Intel(R) Core(TM) i5-13450HX @ 2.40GHz
- **Memory (RAM):** 24.0 GB
- **Programming Language:** Python 3.13.7
- **Compiler:** CPython interpreter used directly

2 Experiments

2.1 Experiment 1: Push-Only Sequence

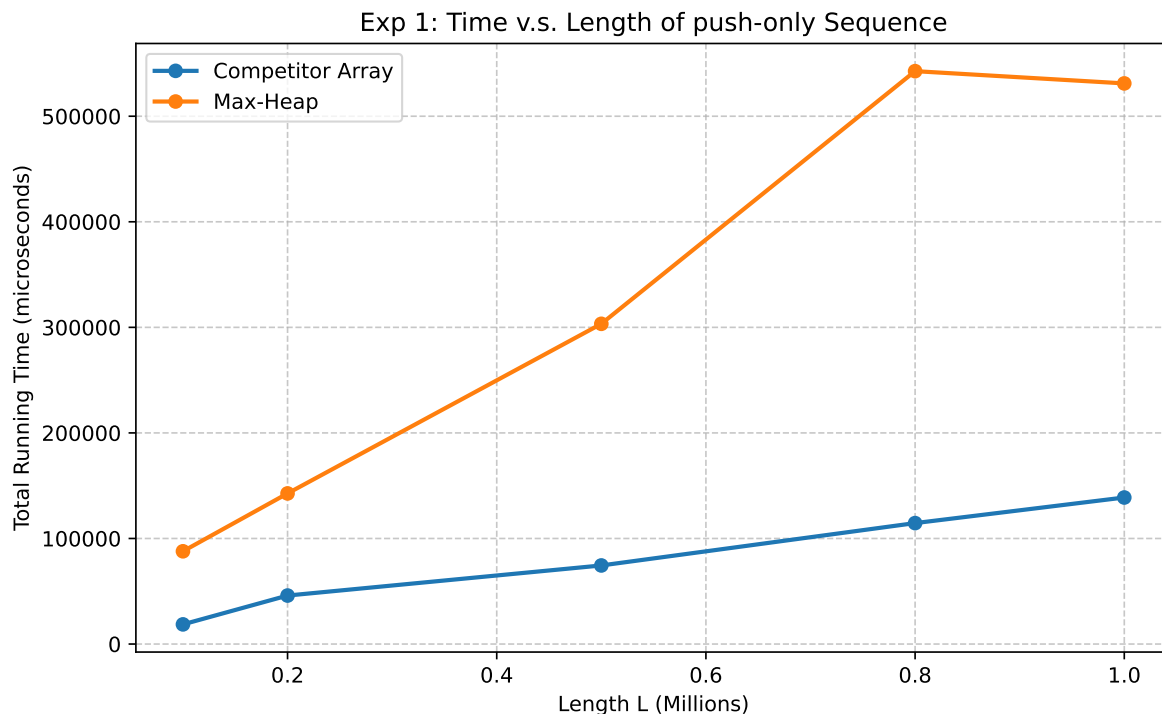


Figure 1: Total running time versus sequence length L for push-only sequences.

Analysis: When we only push items, the Competitor Array is much faster. Adding an item to the end of an array takes $O(1)$ time, so doing it n times takes $O(n)$ total time. On the other hand, every time we push an item into the Max-Heap, it has to run a `bubble_up` operation to find its correct spot. That takes $O(\log n)$ time per item, pushing the total time to $O(n \log n)$. The graph clearly shows this reality wherein the array's $O(n)$ time easily beats the heap's $O(n \log n)$ time when we are strictly doing insertions.

2.2 Experiment 2: Varying `getTop` Percentage

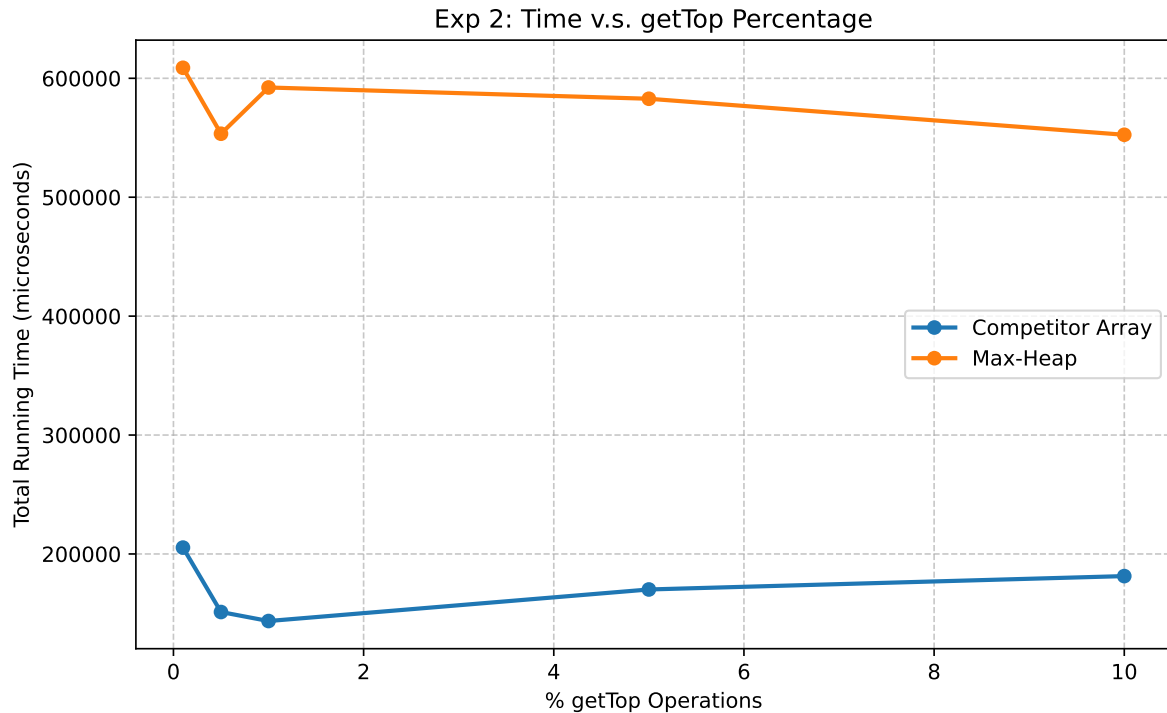


Figure 2: Total running time versus expected percentage of `getTop` operations.

Analysis: In this experiment, we use `getTop` operations. Getting the top item is instant $O(1)$ time for both data structures. Since the array's `push` operation is also $O(1)$, swapping some pushes for `getTop` operations doesn't really change the array's overall workload, which is why its line stays mostly flat. However, the Max-Heap's `push` is a slower $O(\log n)$ operation. As the percentage of `getTops` goes up, the heap is doing more instant $O(1)$ operations and fewer slow $O(\log n)$ pushes. That is why the heap's total running time drops as the percentage increases.

2.3 Experiment 3: Varying pop Percentage

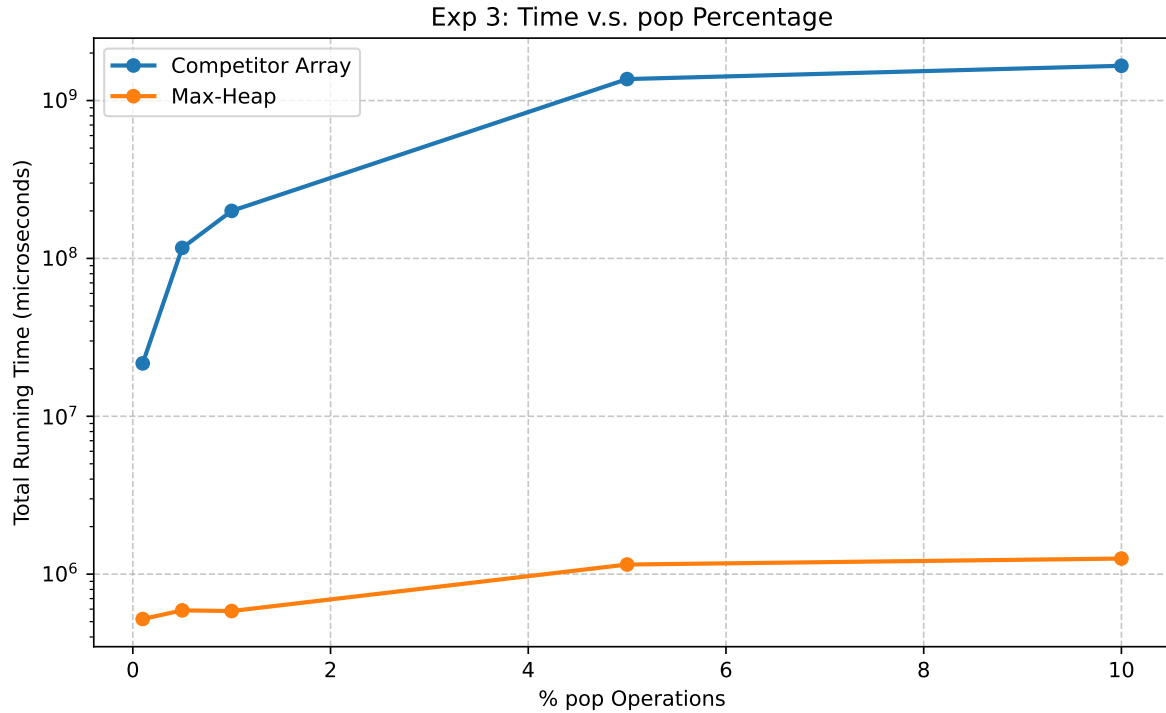


Figure 3: Total running time (Logarithmic Scale) versus expected percentage of pop operations.

Analysis: This experiment exposes the array's biggest weakness. When you call `pop` on the array, it removes the maximum value but then has to scan every single remaining item to find the *new* maximum. That takes $O(n)$ time. Hence, forcing the array to do an $O(n)$ scan thousands of times absolutely reduces its performance, resulting in the massive time spike on the graph. The Max-Heap handles a `pop` by simply moving the last item to the root and running `bubble_down`. That only takes $O(\log n)$ time. Which is why the heap completely dominates here because of its design to handle extractions efficiently.

2.4 Experiment 4: Heapify v.s. Push-One-by-One

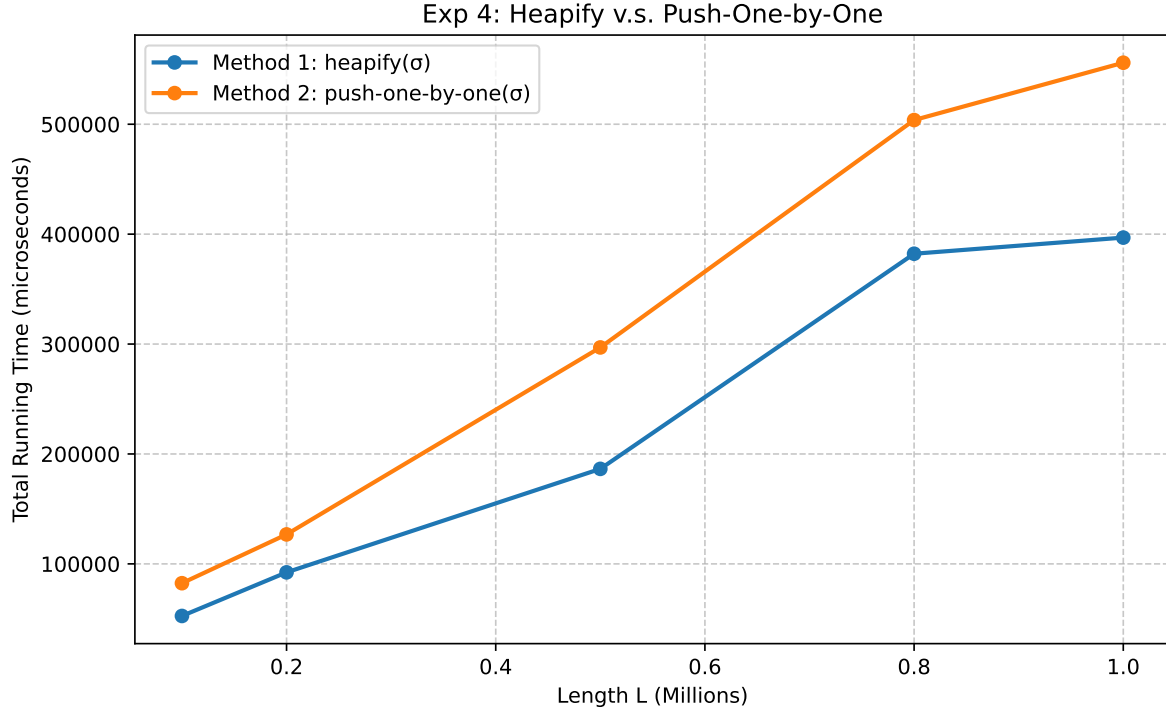


Figure 4: Comparison of total running time between `heapify` and `push-one-by-one` methods.

Analysis: Building a heap by pushing items one by one means we are doing an $O(\log n)$ operation for every single element, which results in an overall time of $O(n \log n)$. (`Heapify`) is mathematically efficient since it dumps all the items into the array first, then works backwards to `bubble_down`. Because the vast majority of nodes in a tree are at the very bottom and don't have to move down very far, the math balances out to a highly efficient $O(n)$ time complexity. As the graph shows, building the tree all at once is significantly faster than building it one by one.

3 Conclusion

Conclusion: The experimental data perfectly support the theoretical time complexities. If a program only needs to insert items and look at the maximum value, the `Competitor Array` is an incredibly fast, lightweight option. However, the moment the program actually needs to extract and remove that maximum value by using (`pop`), the array's $O(n)$ search time makes it unscalable and inefficient for large datasets. The `Max-Heap` is the objectively better choice for real-world use-cases because it keeps all operations running smoothly at $O(\log n)$. Finally, the data also proves that if we already have our full dataset upfront, using `heapify` to build the structure in $O(n)$ time is always better than inserting elements one by one.